Voda - Division of Work & Timeline

Week 1 — Foundation (15 hrs)

Shared:

- Set up GitHub org repo, monorepo structure (/backend, /mobile, /web), .env templates, branching rules
- Agree on and write the Prisma schema together (User, Store, Product, Variant, Order, OrderItem). Run first migration. Write seed script with 2 stores, 10 products, 4 test accounts (one per role)

Narayana (15 hrs):

- Auth API: POST /auth/register, /auth/login, GET /auth/me (bcrypt + JWT)
- Expo project scaffold: React Navigation shell, Zustand skeleton, Axios client
- Login + Register screens wired to live auth API, JWT stored in expo-secure-store

Ayaana (15 hrs):

- Products API: GET /products (with filters), GET /products/:id with variants and store info
- HomeScreen (trending grid) and ProductDetailScreen (images, size selector, hardcoded ETA)
- React Query setup for data fetching

End of Week 1 milestone: both devs can log in on the mobile app and see real products from the database.

Week 2 - Core order flow end-to-end

Narayana (14 hrs):

- Orders API: POST /orders, GET /orders/:id, PUT /orders/:id/status
- Socket.io setup: room joining on auth (user:{id}, store:{id}, order:{id}), emit new_order to store room on order creation
- CartScreen (Zustand), CheckoutScreen (mock payment), OrderConfirmScreen — full checkout flow calling the live API

Ayaana (14 hrs):

- Store API: GET /store/orders, PUT /orders/:id/status (mark picked)
- Store web dashboard (React + Vite): joins store socket room, shows incoming orders in real time, mark-as-picked button
- SearchScreen with filters and Notify Me flow (POST /products/:id/notify)

Shared (1 hr):

- Integration test: Customer places order on mobile → Store dashboard receives it live → Customer sees status update. Fix blockers together.

End of Week 2 milestone (Sprint 1 done): small but complete working flow across all layers. Customer, Store, and real-time socket events all confirmed working.

Week 3 - Runner, Rider & live location

Narayana (13 hrs):

- Runner API: GET /runner/orders, status transitions PENDING → RUNNER_ASSIGNED → COLLECTED → HANDED_TO_RIDER
- Runner app screens: RunnerDashboard, AcceptOrderScreen, CollectionScreen, HandoverScreen (minimal UI, list + action buttons)
- Try & Buy backend: on ARRIVED set tryTimerEnd = now + 10 min, emit try_timer_start to order room, POST /orders/:id/outcome (KEEP or RETURN), RETURNING → RETURNED → REFUNDED transitions

Ayaana (13 hrs):

- Rider API: GET /rider/orders, transitions HANDED_TO_RIDER → OUT_FOR_DELIVERY → ARRIVED
- Rider app screens: RiderDashboard, PickupScreen, DeliveryScreen with live GPS emit every 5s via expo-location
- LiveTrackingScreen for customer: status timeline, rider info, map placeholder. Backend relays rider_location as rider_location_update to customer room. order_update emitted on every status change.

Shared (2 hrs):

- Integration test: run the full 4-actor chain and verify the complete status machine has no dead ends

End of Week 3 milestone: all 4 actors interacting live. Order travels from PENDING to ARRIVED with real socket events throughout.

Week 4 - Try & Buy UI, polish & demo (15 hrs)

Narayana (12 hrs):

- TryBuyScreen: 10-min countdown from tryTimerEnd received via socket, Keep → payment finalised, Return → refund screen, auto-return on timeout
- Backend deployment: Node server to Railway, update all environment variables, smoke test against production

Ayaana (12 hrs):

- Store inventory + settlement: GET/PUT /store/inventory, InventoryManager screen, GET /store/settlement, SettlementView with earnings breakdown (API + UI)
- Bug fixes: loading spinners and error states on every screen, navigation consistency

Shared (3 hrs):

- Demo rehearsal: run the full 6-act demo on real devices (Order → Store notify → Runner collect → Gate handoff → GPS delivery → Try & Buy → Keep or Return). Fix any last blockers.
- 3 hrs buffer reserved for spillover

End of Week 4 milestone (final): all actors functional, realtime updates working, full order lifecycle, Try & Buy timer, live GPS, return/refund flow. Demo-ready.

What we need before starting

1. Supabase project credentials (DATABASE_URL) - create a free project at supabase.com
2. Map decision - react-native-maps or a simple placeholder for the prototype?
3. Payment scope - mock confirmation screen or Stripe test mode?
4. Product images - real images or Cloudinary placeholders in seed data?